

5 Security Protocols 安全协议

Review

- Encryption
- Message authentication code
- Digital signature
- DH key agreement
- Symmetric/Private-key/Secret-key/Shared-key cryptography
- Asymmetric/Public-key cryptography

Outline

- Overview of security protocols
- Key establishment protocols
- More protocols

Learning Objective

- Understand key establishment protocols.
 - Be able to identify and illustrate obvious attacks to a given protocol, and fix the protocol
- Be able to design security protocols for given scenarios.

Overview of Security Protocols

Security Protocol Introduction

- Understand key establishment protocols.
理解关键建立协议。
- Be able to identify and illustrate obvious attacks to a given protocol, and fix the protocol
能够识别并说明针对给定协议的明显攻击，并修复该协议。
- Be able to design security protocols for given scenarios.
能够为特定场景设计安全协议。

Participants in Security Protocols 协议的参与者

正常情况下是有一堆正在通话的两个用户，发送者和接收者。但是在复杂的网络环境中，在这两个人通信的时候，可能还有一些潜伏在中间的监听者（cheater）等待窃取这两个人的对话信息。

潜在的“坏人” (The bad guys):

- **Eve (窃听器):** 仅进行被动监听，试图获取秘密信息。
- **Mallory (攻击者):** 具有主动恶意，可能会拦截、修改或删除消息。
- **内部作弊:** 有时 Alice 或 Bob 本身也可能试图违反协议规则。

Trust Arbitrator 信任仲裁员

当 Alice 和 Bob 彼此不完全信任或没有预共享密钥时，通常需要第三方。

Trent: 一个被所有合法参与者共同信任的、无利害关系的第三方。

作用与优缺点:

- **优点:** 仲裁者的存在往往能简化协议的设计。
- **缺点:** 会增加系统的额外开销 (Overhead)，并且可能会限制协议的适用范围 (例如，如果 Trent 宕机，整个协议就无法进行)
- A disinterested third party trusted by all legitimate participants
所有合法参与者都信任的无利益第三方
- Arbitrators often simplify protocols, but add overhead and may limit applicability
仲裁者通常简化协议，但会增加开销并可能限制其适用范围。

Goals of Security Protocols 安全协议的目标

- Each protocol is intended to achieve some very particular goal
每种协议的设计都旨在实现特定的目标。
 - Like setting up a key between two parties
例如为通信双方创建一个安全的密钥
- Protocols may only be suitable for that **particular** purpose
协议可能仅适用于特定目的。
- Important **secondary** goal is minimalism
次要目标 (极简主义 Minimalism)：一个好的协议应尽可能高效，包括：
 - Fewest possible messages
发送最少的消息数量
 - Least possible data
传输最少的数据量
 - Least possible computation
进行最少的计算工作

Key Establishment Protocols

Key Establishment Protocols

- Often, we want a different encryption key for each communication session
我们通常希望每次通信 (即一个“会话”) 都使用不同的加密密钥。
- How do we get those keys to the participants?
如何在双方可能从未通信过的情况下，安全且快速地分发这些密钥？
 - Securely
 - Quickly
 - Even if they've never communicated before

(Session: period of time over which parties are willing to maintain state 指双方愿意维持通信状态的一段时间)

- Key establishment is a cryptographic mechanism that provides two or more parties communicating over an open network with a shared secret key.
密钥建立是一种加密机制，它为两个或多个在开放网络上通信的参与方提供共享的秘密密钥。
- There are **TWO** Categories:

- **Key transport protocols:** In a key transport protocol, the shared secret key is created by one party and securely transmitted to the second party. (更推荐)

密钥传输协议：由其中一方创建秘密密钥，并将其安全地发送给另一方。

- **Key agreement (or exchange) protocols:** In a key exchange protocol, both parties contribute information which is used to derive the shared secret key.

密钥协商/交换协议：双方都贡献一部分信息，共同推导出一个共享的秘密密钥。

Properties of key establishment protocols 密钥建立协议的属性

- (implicit) Key authentication

(隐式) 密钥认证

- One party is assured that no other party aside from a specifically identified second party (and possibly additional identified trusted parties) may gain access to a particular secret key.

一方确信，除特定指明的第二方（可能还包括其他指定的可信方）外，任何其他方都无法获取特定的密钥。

- Confidentiality + Integrity

保密性 + 完整性

- Authenticated Key Establishment (AKE) protocol

认证密钥建立 (AKE) 协议

- Key confirmation

密钥确认

- One party is assured that a second party actually has possession of a particular secret key

一方确信另一方实际以及持有了该密钥

- Explicit key authentication

显式密钥认证

- (implicit) Key authentication + key confirmation

即“隐式密钥认证” + “密钥确认”

Session Key 会话密钥的特性

- Key establishment protocols result in shared secrets which are typically called, or used to derive, session keys.

派生性：密钥建立协议产生的共享秘密通常被直接称为会话密钥，或者用来进一步推导出具体使用的会话密钥

- Ideally, a session key is an ephemeral secret restricted to a short time period.

临时性 (Ephemeral)：理想情况下，会话密钥应该是临时的，仅在很短的时间内有效，过期即作废。

Key Transport Protocol 密钥传输协议

Key transport with private-key cryptography 对称加密下的密钥传输

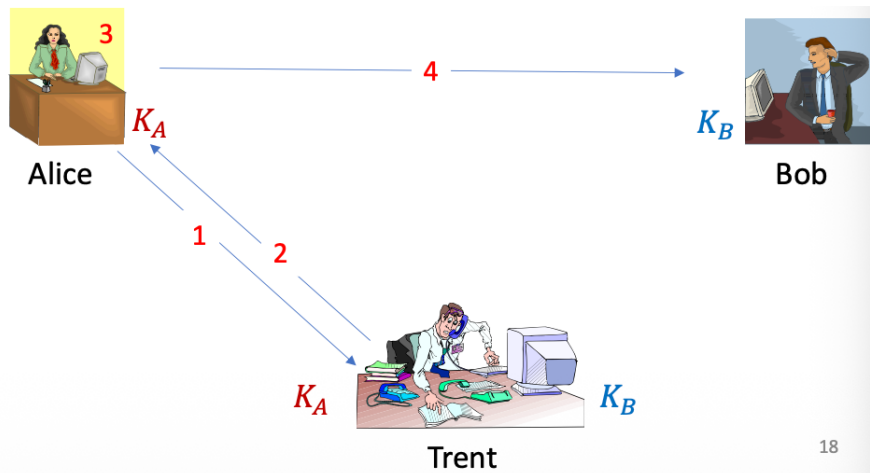
在这个场景中，Alice 和 Bob 不认识，但他们都信任第三方 **Trent**，并分别与 Trent 预共享了密钥 K_A 和 K_B

- Alice and Bob want to talk securely with a new key
- They both trust Trent
 - Assume Alice & Bob each share a key with Trent
- How do Alice and Bob get a shared key?

Protocol

协议流程: Trent 生成一个会话密钥 K_{AB} , 分别用 K_A 和 K_B 加密后发给 Alice, Alice 再把给 Bob 的那部分转发过去。

- Initially, Alice and Trent share K_A , Bob and Trent share K_B
- Key Establishment
 1. Alice -> Trent: Alice requests for a session key with Bob
 2. Trent -> Alice: $\{K_{AB}\}_{K_A}, \{K_{AB}\}_{K_B}$
 3. Alice: $K_{AB} \leftarrow \{K_{AB}\}_{K_A}$
 - Alice -> Bob: $\{K_{AB}\}_{K_B}$
 4. Bob: $K_{AB} \leftarrow \{K_{AB}\}_{K_B}$



18

What has the protocol achieved?

- Alice and Bob both have a new session key
- The session key was transmitted using keys known only to Alice and Bob
- Both Alice and Bob know that Trent participated
- But there are vulnerabilities
- What if the initial request was grabbed by Mallory? Suppose Mallory is also a user of the system and share K_M with Trent.

The man -in-the-middle attack



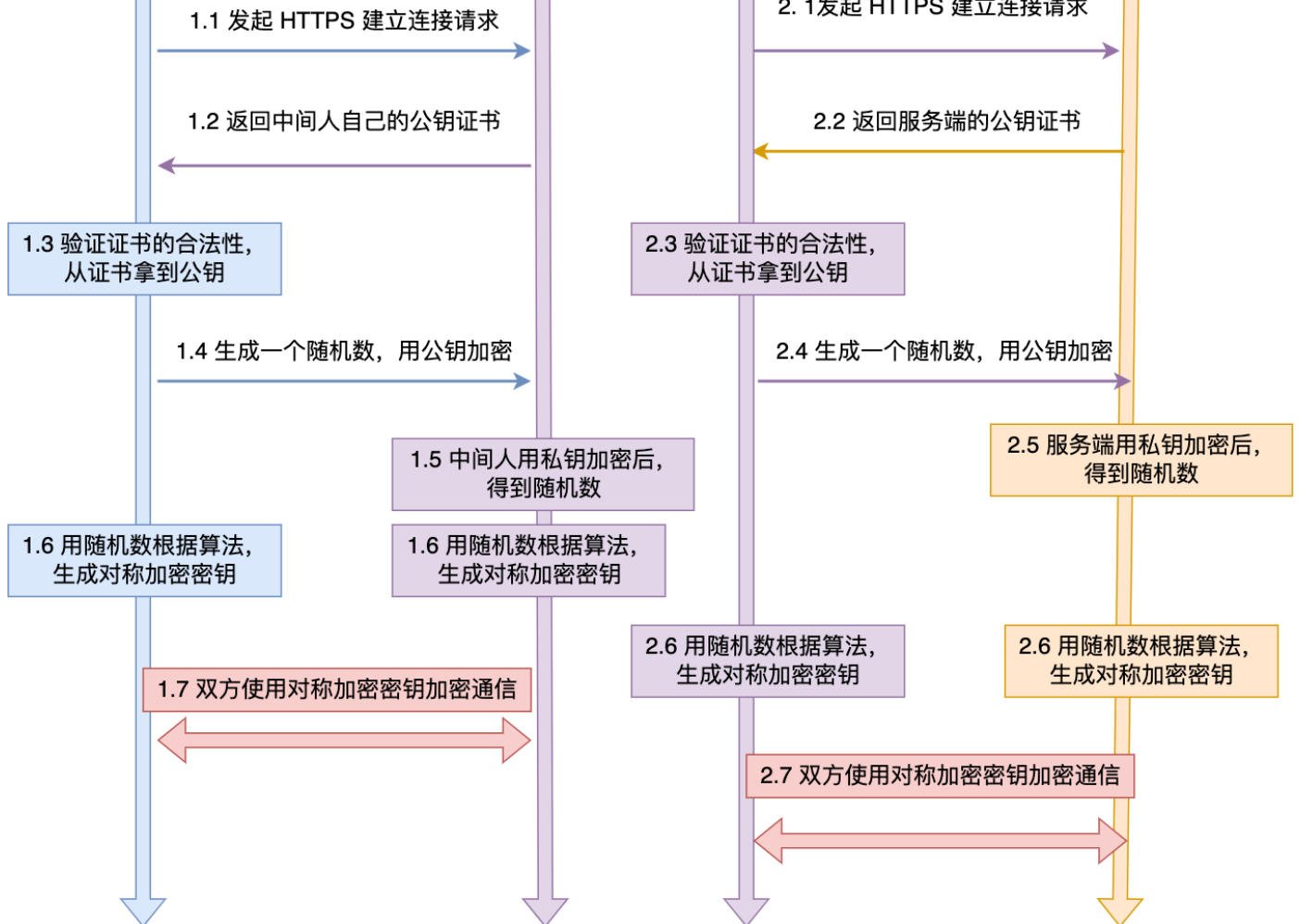
客户端



中间人服务器



服务端





Alice

K_A



Mallory

K_M



Bob

K_B



Trent

- 对称加密中间人攻击流程：攻击重点是篡改原始请求消息
 - 手法：攻击者 Mallory 拦截 Alice 的请求，将其篡改为“Alice 想跟 **Mallory** 通话”。
 - 后果：Trent 误以为是合法请求，生成了 Alice 和 Mallory 之间的密钥。Mallory 随后再伪造身份与 Bob 建立另一个密钥。
 - 核心原因：最初的请求缺乏身份认证，Trent 无法确认请求是否真的来自 Alice 且未被篡改。
- Initially, Alice and Trent share K , Bob and Trent share $K\#$, Mallory and Trent share K_M
 1. Alice -> Trent: Alice requests for a session key with Bob
 2. Mallory replaces the message as following
 - Mallory -> Trent: Alice requests for a session key with Mallory
 3. Trent -> Alice: $\{K_{AM}\}_{K_A}, \{K_{AM}\}_{K_M}$
 4. Alice: $K_{AM} \leftarrow \{K_{AM}\}_{K_A}$
 - Alice -> Bob: $\{K_{AM}\}_{K_M}$
 5. Mallory: $K_{AM} \leftarrow \{K_{AM}\}_{K_M}$
 6. Mallory -> Trent: Mallory requests for a session key with Bob
 7. Trent -> Mallory: $\{K_{MB}\}_{K_M}, \{K_{MB}\}_{K_B}$
 8. Mallory: $K_{MB} \leftarrow \{K_{MB}\}_{K_M}$ and replace message instep 4 with $\{K_{MB}\}_{K_B}$
 9. Bob: $K_{MB} \leftarrow \{K_{MB}\}_{K_B}$

Defeating the man-in-the-middle attack 击败中间人攻击

- Problems:
 - Lacking authentication
缺少合理的认证手段
- Minor changes can fix that problem
或许只需要一些微小的改动就可以解决上面的问题
 - using MAC
使用MAC地址校验，因为每个设备都有独一无二的MAC地址

- including timestamps/counter and identity in the messages

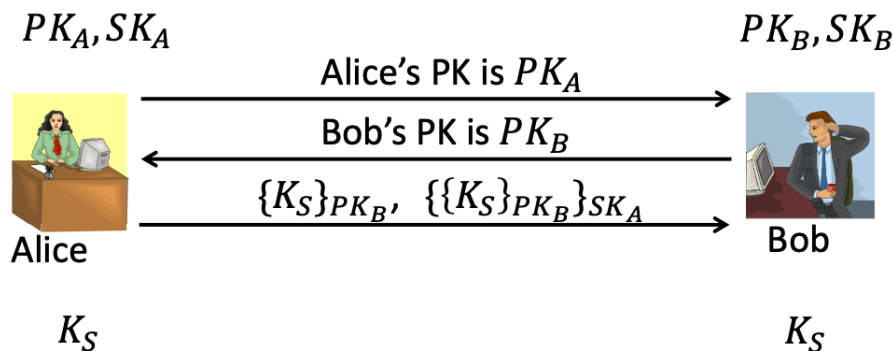
包括消息中的时间戳/计数器和身份信息

Key transport with public key cryptography 公钥加密下的密钥传输

这个场景不需要 Trent, Alice 和 Bob 直接交换公钥 (PK)。

- With no trusted arbitrator
- Alice sends Bob her public key
Alice 直接发送给 Bob 她的公钥
- Bob sends Alice his public key
Bob 直接发送给 Alice 他的公钥
- Alice generates a session key and sends it to Bob encrypted with his public key, signed with her private key
爱丽丝生成一个会话密钥，并用鲍勃的公钥加密后发送给他，同时使用自己的私钥进行签名。
- Bob decrypts Alice's message with his private key
鲍勃用他的私钥解密了爱丽丝的消息。并且可以使用之前接收到的 Alice 的公钥来验证 Alice 用私钥加密的信息。
- Encrypt session with shared session key
使用共享会话密钥加密会话

Basic key transport using PKC



Bob verifies the message came from Alice
Bob extracts the key from the message

Protocol

1. Alice $\rightarrow$ Bob: PK_A
2. Bob $\rightarrow$ Alice: PK_B
3. Alice $\rightarrow$ Bob: $c = \{K_S\}_{PK_B}, \sigma_A = \{c\}_{SK_A}$
4. Bob: verifies σ_A and c ; extract K_S from c .

Man-in-the-middle attack

1. Alice $\rightarrow$ Bob: PK_A
2. Mallory $\rightarrow$ Bob: PK_M
3. Bob $\rightarrow$ Alice: PK_B
4. Mallory $\rightarrow$ Alice: PK_M

5. Alice $\rightarrow$ Bob: $c = \{K_S\}_{PK_M}, \sigma'_A = \{c\}_{SK_A}$

6. Mallory extract K_S from c

◦ Mallory $\rightarrow$ Bob: $c' = \{K_S\}_{PK_B}, \sigma'_A = \{c'\}_{SK_M}$

7. Bob: verifies σ'_A and c' ; extract K_S from c'

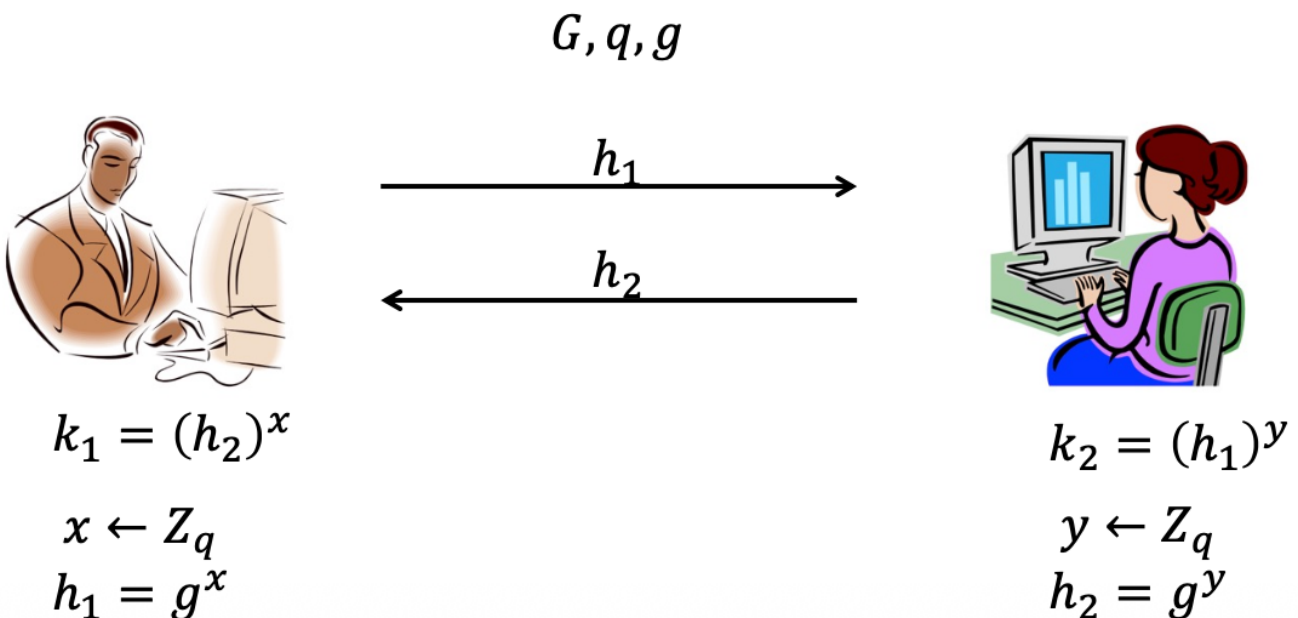
- **公钥加密中间人攻击的手法**：攻击重点是拦截并替换公钥（identity substitution）
- **手法**：Mallory 拦截双方交换的公钥。他把自己的公钥 PK_M 发给 Alice（假装是 Bob 的），同时把 PK_M 发给 Bob（假装是 Alice 的）。
- **后果**：Alice 用 Mallory 的公钥加密了密钥 K_S ，Mallory 拦截后可以用自己的私钥解密提取密钥，然后再用 Bob 的公钥加密发给 Bob。
- **核心原因**：虽然使用了加密和签名，但公钥本身没有绑定身份。Alice 无法确认手里的“Bob 的公钥”到底是不是真的属于 Bob。

Solution

- PKI: public-key infrastructure
 - CA (certificate authority) issues public-key certificates
CA（证书颁发机构）签发公钥证书。
- Identity-based cryptography
基于身份加密技术
 - the identity is (or can be used to derive) the public-key
身份是（或可用于推导出）公钥。

Key Exchange Protocols

Diffie-Hellman key agreement 密钥协商



G : cyclic group

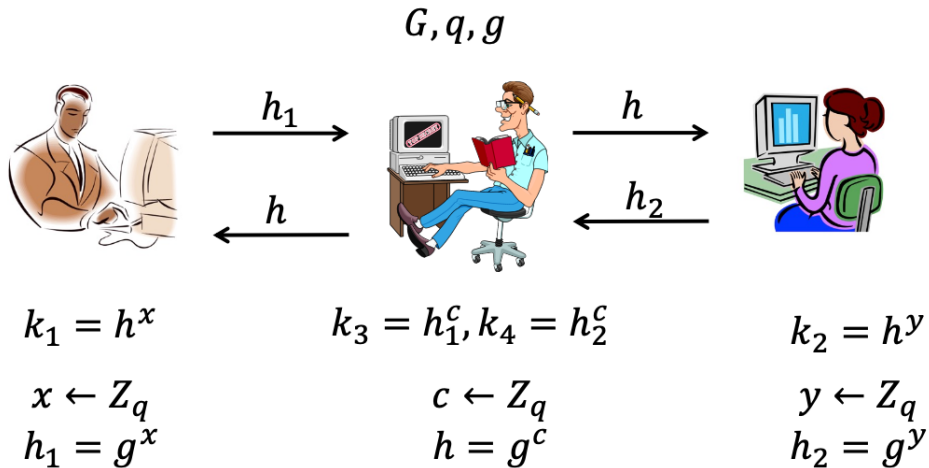
q : prime, order of G

g : generator of G

这是最经典的密钥交换协议，其安全性基于离散对数难题。

- **参数准备**：双方共享循环群 G 、阶 q 和生成元 g 。
- **交互过程**：
 1. **Alice** 选择随机数 x ，计算并发送 $h_1 = g^x$ 。
 2. **Bob** 选择随机数 y ，计算并发送 $h_2 = g^y$ 。
- **计算密钥**：Alice 计算 $(h_2)^x$ ，Bob 计算 $(h_1)^y$ 。由于 $(g^y)^x = (g^x)^y = g^{xy}$ ，双方得到了相同的共享密钥。
- **DDH 问题**：协议的安全性依赖于攻击者无法从 g^x 和 g^y 中分辨出真实的 g^{xy} 。

Man-in-the-middle to (EC)DH DH 协议的中间人攻击



- Reason:
 - Solution:
- Lack of authentication
 - Introduce authentication in the protocol

就像之前的协议一样，基础版的 DH 协议也面临严重的中间人攻击。

- **原因**：缺乏身份认证。Alice 无法确认收到的 $h = g^c$ 是来自 Bob 还是来自攻击者 Mallory。
- **攻击过程**：Mallory 拦截 Alice 和 Bob 的消息，分别与他们建立两个独立的密钥 (k_1 和 k_2)，从而可以解密、修改并转发两人的通信，而当事人毫无察觉。
- **对策**：必须在协议中引入身份认证机制。

ECDH-based AKE 认证密钥建立 (AKE) 实例

为了解决上述攻击，实际应用中会使用增强版协议，如 **ECDH-based AKE**

IEEE 802.15.6 密码认证关联

这是一个复杂的工业标准实例，通过密码 (Password) 来增强安全性。

- 它使用了 **Nonce (随机值)** 来防止重放攻击。
- 通过多次“安全关联帧” (Security Association frame) 的交互来确认身份并激活密钥。

Simplified Version 简化版流程

- Initialization:
 - A, B share group parameters and password $PW, Q : \{0, 1\}^* \rightarrow G$
 - Key exchange:

1. A: generates SK_A, PK_A , computes $PK'_A = PK_A - Q(PW)$, generates N_A
 - $A \rightarrow B : B, A, N_A, PK'_A$
2. B: generates SK_B, PK_B , generates N_B
 - $A \rightarrow A : A, B, N_B, PK_B$
3. computes $PK_A = PK'_A + Q(PW)$, computes shared secret $K = SK_B \cdot PK_A$, $t_B = MAC(K_x, A, B, N_A, N_B)$
4. A: computes shared secret $K = SK_A \cdot PK_B$, verifies t_B , computes $t_A = MAC(K_x, B, A, N_B, N_A)$
 - $A \rightarrow B : B, A, N_A, PK_A, t_A$
5. B: verify t_A

- **初始化**：双方共享参数和密码 PW 。
- **掩码处理**：Alice 生成临时公钥 PK_A ，但发送的是经过密码打乱的 $PK'_A = PK_A - Q(PW)$ 。这样只有知道密码的 Bob 才能还原真实的公钥。
- **确认 (MAC)**：双方在计算出共享秘密 K 后，会交换 **MAC (消息认证码)**。
 - Bob 发送 $t_B = MAC(K_x, \dots)$ 给 Alice。
 - Alice 验证 t_B 后发送 t_A 给 Bob。
- **结果**：这种“密钥确认”步骤确保了双方不仅算出了密钥，而且确信对方也算出了正确的密钥。

Lab 5中的ECDH任务：

预先存在：在协议开始之前，A 和 B 已经通过物理接触或其他安全渠道（如用户手动输入、设备出厂设置）共享了一个共同的密码 PW 。

混淆与还原：

- **A 端 (发送方)**：利用一个映射函数 Q 将密码转换为椭圆曲线上的一个点 $Q(PW)$ 。A 发送的不是原始公钥 PK_A ，而是计算 $PK'_A = PK_A - Q(PW)$ 。
- **B 端 (接收方)**：B 收到 PK'_A 后，利用手中持有的相同密码计算出相同的 $Q(PW)$ ，然后通过加法运算 $PK_A = PK'_A + Q(PW)$ 还原出 A 的真实临时公钥。
- **认证逻辑**：如果 B 手中的密码不正确，还原出的 PK_A 就是错误的，后续计算出的共享密钥 K 也无法通过 MAC（消息认证码）的校验，从而确保了只有合法持有密码的双方才能解密和使用该公钥。

More Protocols

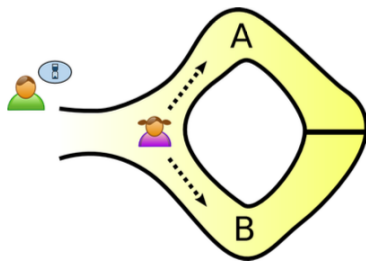
Zero-Knowledge Proof (ZKP) 不必理解技术，只了解概念

- Encryption provides secrecy in a very all-or-nothing sense:

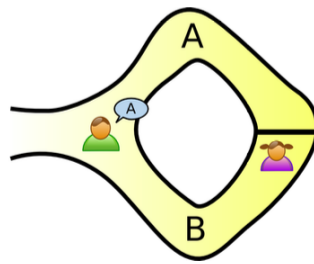
加密在某种意义上提供了一种非此即彼的保密性：

 - a certain party (with the key) could either completely decrypt the message, learning the message contents, or cannot do anything with the message at all.

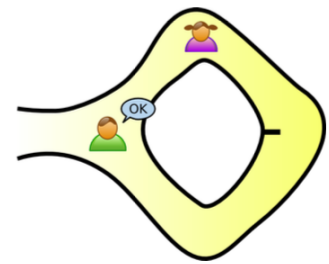
持有密钥的一方要么能够完全解密信息，了解信息内容，要么就完全无法处理该信息。
 - ZKPs allow one party (called a prover) to convince another party (called a verifier) that the prover **“knows”** some secret, **without revealing** anything else about the secret.
- 零知识证明允许一方（称为证明者）说服另一方（称为验证者）自己“知道”某个秘密，而无需透露关于该秘密的任何其他信息。



Peggy randomly takes either path A or B, while Victor waits outside.



Victor chooses an exit path.



Peggy reliably appears at the exit Victor names.

- More formally, say that both parties hold a function f , and a value $y = f(x)$. The prover may want to convince the verifier that it "knows" an x such that $y = f(x)$ without revealing anything else about x .

Three Properties:

- **Completeness:** If the prover "knows" x , it will always convince an honest verifier to accept.
- **Soundness:** If the verifier accepts the proof, then prover must really "know" x .
- **Zero-knowledge:** The verifier "learns nothing" about x , apart from the fact that $y = f(x)$, as a result of interacting with the prover.

Schnorr's protocol

Schnorr's protocol for proving knowledge of discrete logs

- Parameters: G, g, q
- Prover holds a secret: $x \in \mathbb{Z}$
- Verifier holds a public value: $y = g^x \in G$
- The prover wants to prove this to the verifier this without revealing anything about else about its secret value x .

Schnorr's protocol for proving knowledge of discrete logs

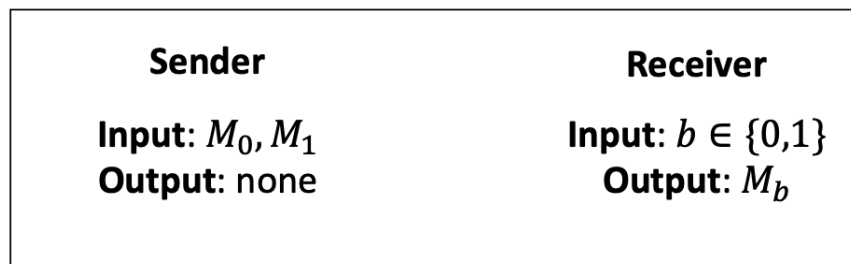
1. Prover picks a random $r \in \mathbb{Z}$, computes $R = g^r \in G$ and sends it to verifier.
 2. Verifier picks a challenge bit $c \in \{0,1\}$ at random and sends it to prover.
 3. Prover sends back a response $z \in \mathbb{Z}$ generated as following: $z = r + c \cdot x$
 - If $c = 0$, set $z = r \in \mathbb{Z}$
 - If $c = 1$, set $z = r + x \in \mathbb{Z}$
 4. Verifier checks if $g^z = R \cdot y$,
- Repeat the process to reduce soundness error.

Oblivious Transfer (OT)

- An oblivious transfer (OT) protocol is a type of protocol in which a sender transfers one of potentially many pieces of information to a receiver, but remains oblivious as to what piece (if any) has been transferred.

不经意传输 (OT) 协议是一种协议类型, 其中发送方将多条潜在信息中的一条传送给接收方, 但对具体传送了哪条信息 (如果有的话) 一无所知。

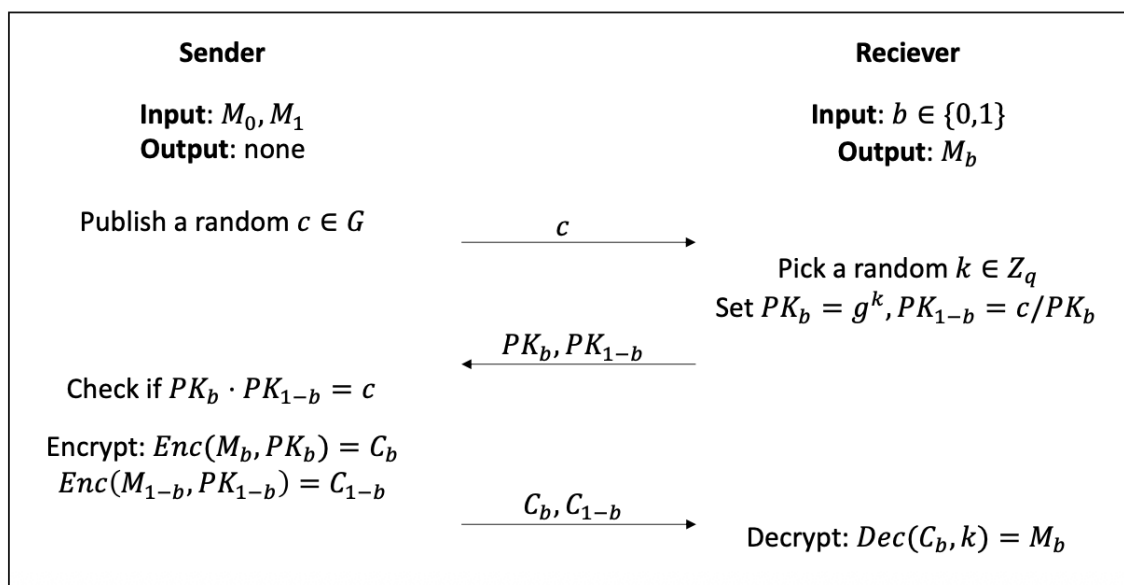
- Simplest 1-out-of-2 OT



1-out-of-2 OT

- Bellare-Micali Construction
- Parameters
 - G, q, g

CDH Assumption

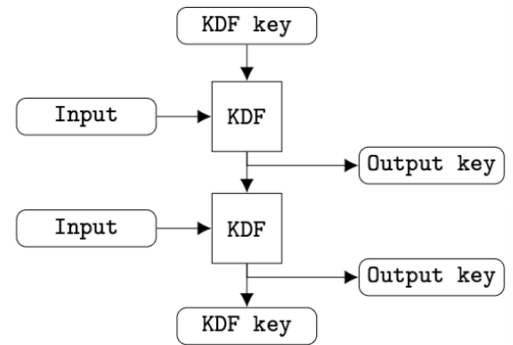
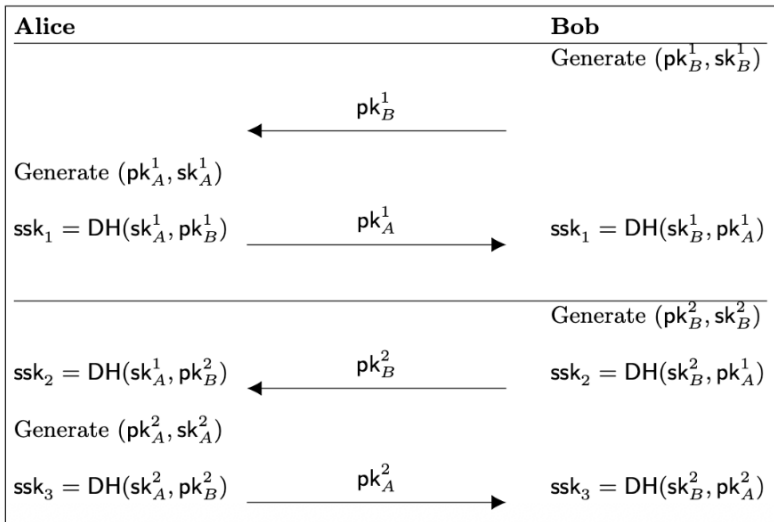


Signal Protocol

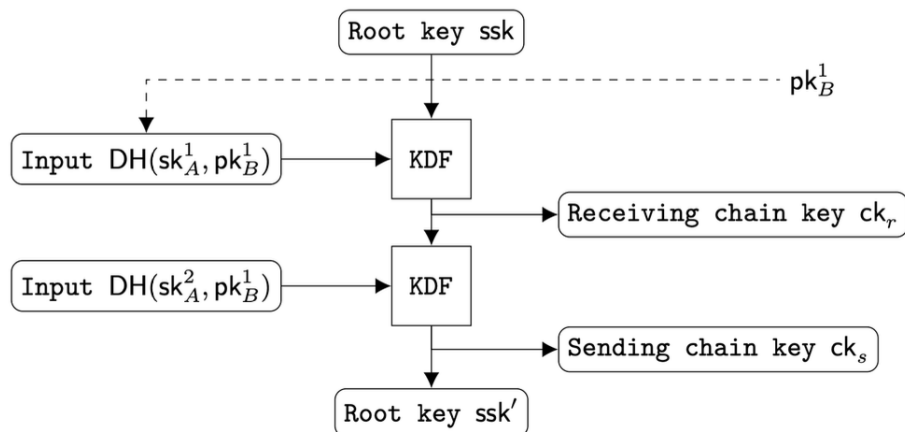
- Developed in 2013 by Open Whisper Systems to provide a secure, asynchronous messaging protocol.
- Powers end-to-end encryption for Signal Messenger . Also adopted by major - platforms like WhatsApp, Google Messages (RCS), and Facebook Messenger
- Core Security Properties
 - **End-to-End Encryption (E2EE):** Only sender and recipient can read messages.
 - **Forward Secrecy:** Past messages remain secure if long-term keys are compromised.
 - **Post-Compromise Security:** The protocol automatically heals itself, securing future messages after a key compromise

Ratcheted Key Exchange

- DH ratchet on protocol level



- DH ratchet from Alice's perspective



The root ssk

- X3DH protocol

How to evaluate a security protocol

- Mathematics proof
- Formal methods: BAN, GNY logics
 - Further reading paper
- Tools
 - Casper: <https://www.cs.ox.ac.uk/gavin.lowe/Security/Casper/>
 - SPIN: <https://spinroot.com/spin/whatispin.html>
 - ProVerif: <https://bblanche.gitlabpages.inria.fr/proverif/>